# Data Storm 7.0 — Team Data-Drifters

## Unlocking Latent Demand: A Three-Tier Lakehouse Pipeline and Causal Inference Framework for Potential-Based Asset Allocation

**Team Data-Drifters** **S Kaarthiraghav** **K A D P Kumarapeli**
*NSBM Green University*

**W M A Kisara Wethmin**

May 16, 2026

**Abstract**

This technical report details the end-to-end analytical and data engineering framework implemented by team **Data-Drifters** to solve the latent demand estimation challenge. Traditional resource allocation relies heavily on historical sales averages, a methodology constrained by supply-side shocks, credit thresholds, and distribution blackouts. We address this by designing a robust, industry-standard Bronze → Silver → Gold Lakehouse architecture, trapping legacy system anomalies, scraping spatial catchment drivers via OpenStreetMap, and engineering an ensemble causal modeling approach. Our final model successfully uncaps right-censored historical volume to predict the maximum monthly purchase potential for 20,000 retail outlets for January 2026.

**Organized By**
Rotaract Club of University of Moratuwa
Rotract Club of University of Colombo Faculty of Science

**Powered By**
OCTAVE – John Keells Group

# 1 Introduction & Business Context

Legacy resource allocation in traditional trade networks frequently falls into the trap of allocating marketing spend, promotional discounts, and cold-chain assets (coolers) based purely on historical throughput. This approach mistakes historical sales for true consumer demand, ignoring systemic constraints such as out-of-stock events, delivery caps, and localized credit freezes.

Team **Data-Drifters** addresses this paradigm shift towards **Potential-Based Allocation**. Our scope focuses on a network of 20,000 traditional retail outlets (including corner-shop "kades", local eateries, and pharmacies) spread across four strategic provinces of Sri Lanka (Western, Central, North-Western, and Southern), serviced by 10 distinct distributor networks.

# 2 Data Engineering, Forensics & Hygiene

System data from legacy Sales Force Automation (SFA) and distributor ERPs are inherently noisy, containing human entry shortcuts, automated ghost rows, and transactional gaps due to connectivity blackouts. We built an automated, parameterized data quality framework structured into a classic Medallion architecture.

## 2.1 Lakehouse Pipeline Architecture

The codebase 'code.ipynb' enforces strict isolation between data states:

**Bronze (Raw Ingestion):** Ingests all core flat files ('transactions_history_final.csv', 'outlet_master.csv', 'outlet_coordinates.csv', 'distributor_seasonality_details.csv', and 'holiday_list.csv') as-is into immutable storage formats ('.pkl' and '.csv'), preserving raw lineage and maintaining a 1,000-row audit sample.

**Silver (Sanitized):** Executes deterministic data quality (DQ) checks. Transactions failing assertions are explicitly split and quarantined into 'silver/rejected/' with documented failure reasons, ensuring zero silent drops. Clean records pass into 'silver/'.

**Gold (Enriched Feature Store):** Generates model-ready aggregates at the outlet level ('gold/outlet_features.csv joining spatial signals and historical statistical matrices.

## 2.2 Reusable Data Quality Forensics

Rather than using hardcoded filters, data cleaning is driven by reusable, parameterizable Python functions targeting five system vulnerabilities:

- **Duplicate Check:** Composite keys evaluating unique entries per 'Outlet_ID', 'Date', and 'SKU_ID'.
- **Null Check:** Evaluation of mandatory structural vectors (e.g., coordinates, primary transaction fields).
- **Referential Integrity Check:** Cross-validates all operational transactions against the primary dimension table ('outlet_master.csv').
- **Value Range Check:** Assures physical constraints are non-negative (e.g., volume sold $\geq 0$).
- **Format / Type Check:** Standardizes messy SFA date variations and maps structural categories cleanly.

# 3 External POI Data Acquisition & Scraping

Geospatial surroundings serve as highly predictive proxies for consumer foot traffic and spatial catchment capacity. To enrich internal profiles, we built an external data pipeline.

## 3.1 Infrastructure and Scraping Setup

Using public spatial APIs via the **Overpass API (OpenStreetMap)**, we scraped point-of-interest (POI) signals mapped to the exact coordinate pairs provided in 'outlet_coordinates.csv'. To minimize API thrashing, ensure stability, and prevent rate-limiting overhead during the 36-hour window, the scraper implements a localized key-value cache layer ('gold/poi_raw.pkl').

## 3.2 Catchment Mapping and Spatial Features

We targeted specific catchment drivers associated with high ambient beverage consumption: schools, bus stands, railway hubs, hospitals, local markets, and tourist attractions. High-velocity spatial features were engineered using two methods:

1. **Density Counts:** Aggregating total POI counts within an outlet-specific radial buffer.
2. **Proximity Metrics:** Computing exact distance matrices to the closest landmark using a **BallTree** structure configured with the Haversine metric to account for spatial earth curvature.

Columns are appended to the Gold layer with 'poi_' and 'nearest_' tags.

# 4 Methodology & Causal Base Math

The core analytical challenge is that the target variable, **Latent Maximum Monthly Purchase Potential**, is hidden and right-censored. The historical transactional volume $V_{\text{hist}}$ does not represent true unconstrained demand $D$, but rather the intersection of demand with a systemic supply ceiling $C$.

## 4.1 Isolating the Unconstrained Demand Curve

To uncap the demand curve without an explicit ground truth target ($y$), we isolate historical records that did not face operational barriers.

- **Constraint Labeling:** We track structural variance by computing a censorship indicator using the coefficient of variation and a historical censorship metric per outlet. Outlets showing stable, regular replenishment without sharp declines are flagged as 'is_constrained = 0' (Unconstrained Peers).
- **Base XGBoost Modeling:** We fit an array of **XGBoost Regressors** exclusively on these unconstrained channels to learn the true, clean relationship between an outlet's environmental context (POI densities, distance vectors, SKU breadth, and regional properties) and its maximum throughput.

### 4.2 Ensemble Ceiling Approach

To robustly capture an upper-bound potential ceiling rather than a standard average tendency, we construct an ensemble of three models:

1. **Mean Trend Model:** Standard objective XGBoost to model central tendency.
2. **Upper Quantile Model:** A specialized XGBoost model optimized with a quantile loss objective ($q = 0.90$) to project high-end capacity.
3. **Empirical Peer Ceilings:** Deterministic aggregations computing maximum historical benchmarks grouping outlets strictly by their stratification tier: '(Outlet_Type, Outlet_Size)'.

The models are ensembled via a weighted combination optimized using 5-fold cross-validation on the unconstrained set.

### 4.3 Temporal Adjustment and Enforced Guardrails

To forecast the target month of **January 2026**, the ensemble output is scaled by a distributor-level seasonal multiplier, derived from 'distributor_seasonality_details.csv', alongside localized growth trends and holiday factors. Finally, safety guardrails (empirical floors and ceilings) are applied to bound the output, preventing extreme projections and ensuring business viability.

## 5 Generative AI Transparency Log

In alignment with the evaluation criteria, this section explicitly documents our interaction with LLM models as engineering accelerators during the 36-hour hackathon window.

Table 1: Generative AI Utilization Ledger

| **Pipeline Phase** | **Specific AI Application** | **Verification / Human Modification** |
|---|---|---|
| Data Engineering | Boilerplate syntax generation for reusable parametric null and range checkers. | Manual integration of custom logging structures to route rows to 'silver/rejected/'. |
| POI Scraping | Construction of Overpass QL syntax queries and exponential back-off wrapper loops. | Implemented a localized pickle caching system to bypass redundant API network requests. |
| Statistical Modeling | Brainstorming alternative formulations for the mathematical right-censored demand curve. | Rejected purely probabilistic survival models in favor of an unconstrained peer XGBoost approach. |
| Code Optimization | Profiling Pandas code and rewriting slow operations using vectorized expressions. | Verified computational consistency using code isolation tests before merging. |

**Rigorous Validation Statement:** AI code generations were never trusted blindly. All suggested algorithms underwent local code isolation testing, syntax checking, and evaluation against our validation framework ('output/validation_distribution.png') before pipeline integration.

## 6 Outputs and Repository Artifacts

The full data pipeline produces the following structured files in the repository:

- **Data Ingestion Layer:** Sample data segments preserved under ‘bronze/‘.
- **Quarantine Store:** Anomalous records separated under ‘silver/rejected/‘.
- **Enriched Matrices:** Structured tables: ‘gold/outlet_features.csv‘ and ‘gold/outlet_features.pkl‘.
- **Analytical Diagnostics:** 7 distinct EDA forensic heatmaps and validation plots:
  - ‘gold/eda_01_data_forensics.png‘ (Visual breakdown of legacy system noise)
  - ‘gold/eda_03_censorship_analysis.png‘ (Verification of unconstrained base sets)
  - ‘gold/eda_06_poi_validation.png‘ (Geospatial catchments and proximity verification)
- **Final Potential Submission File:** Saved at ‘output/Data_Drifters_predictions.csv‘, containing the required index parameters: ‘Outlet_ID‘ and the unconstrained potential metric ‘predicted_jan_2026_potential‘.